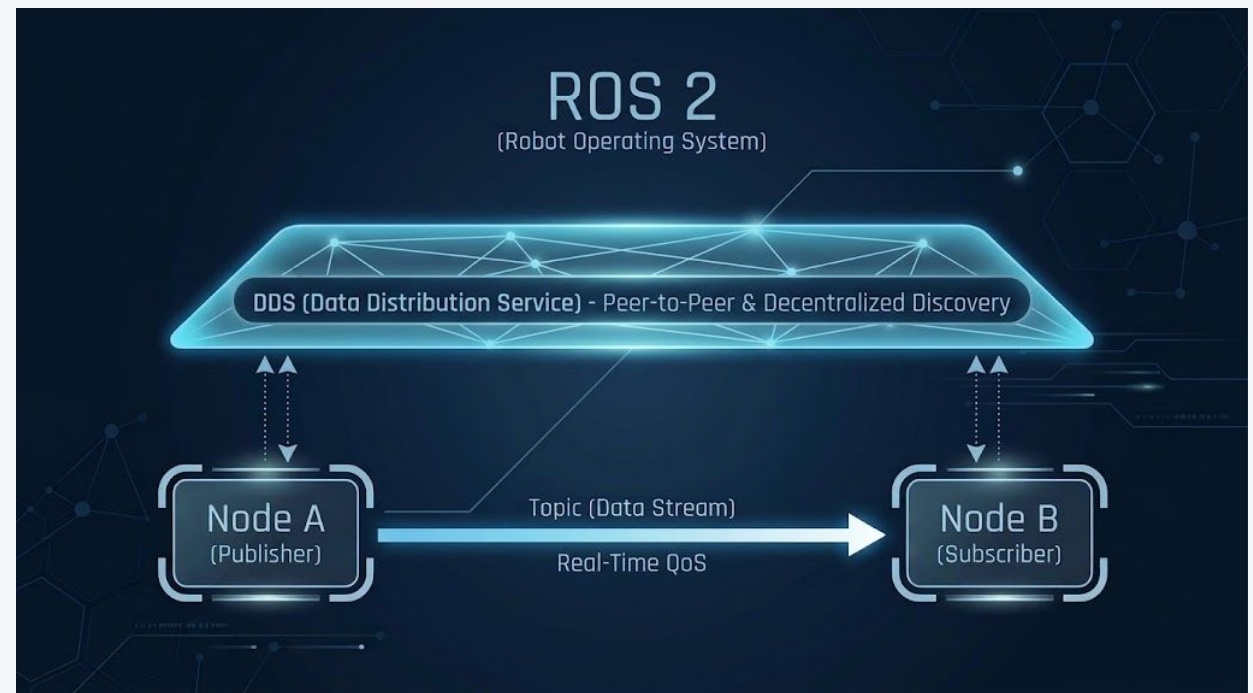
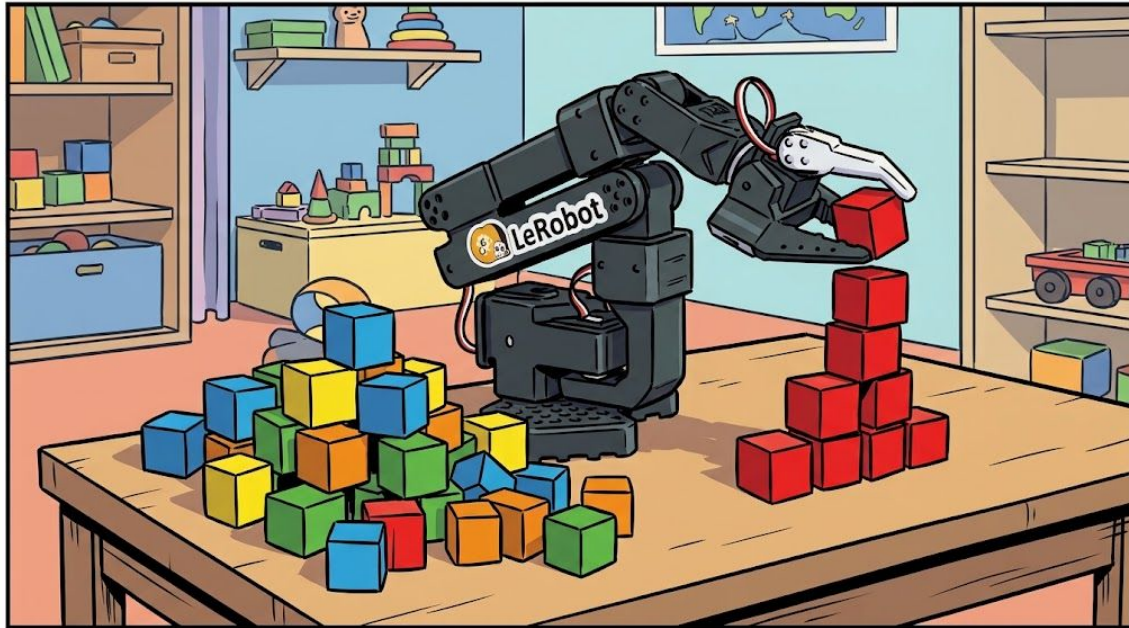


Bootstrap part 2 : Recall

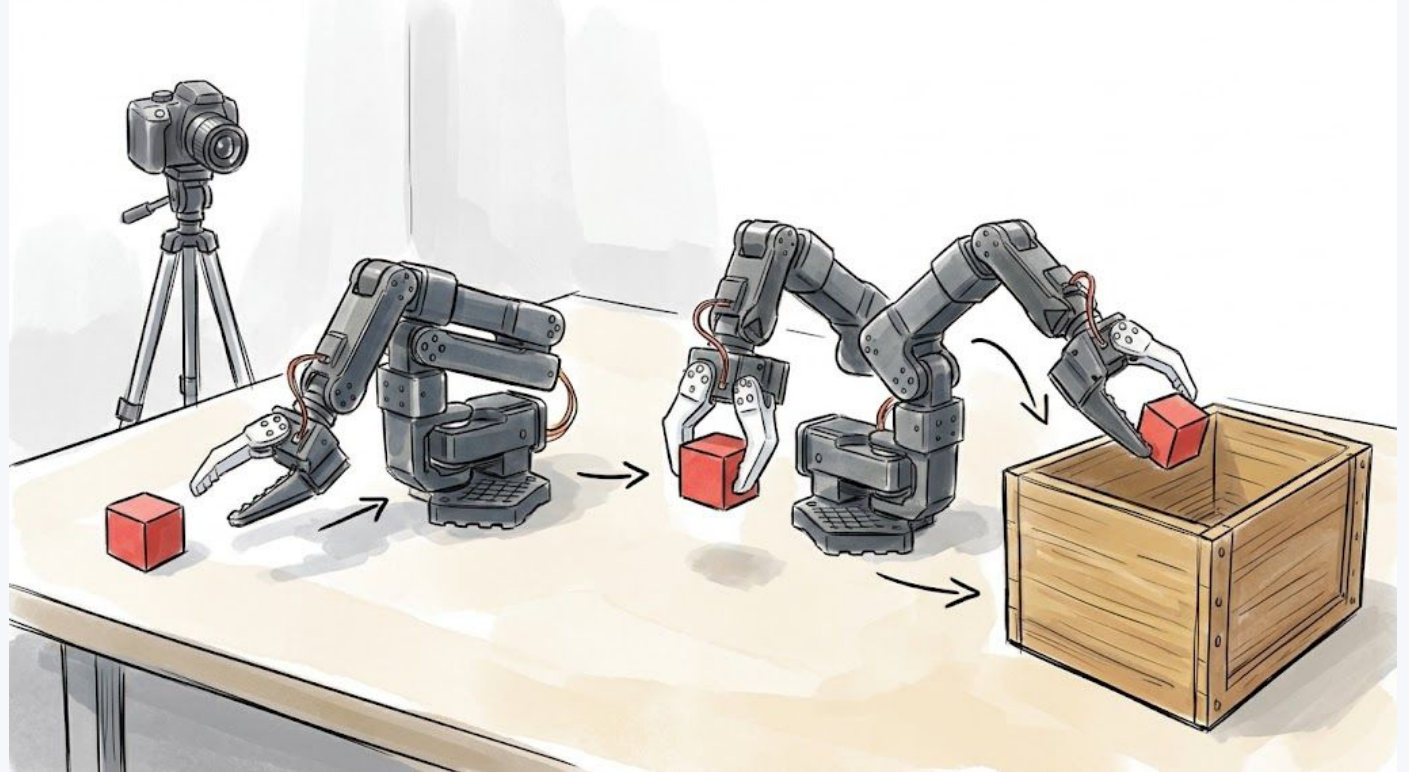


< ROS2 and the robot arm SO101 />



< The goal />

See the object.
Grasp it.
Drop it in the box.



With an external camera, an SO-101 arm and a ROS 2 pipeline you build.

In MuJoCo simulation first, then on the real arm.

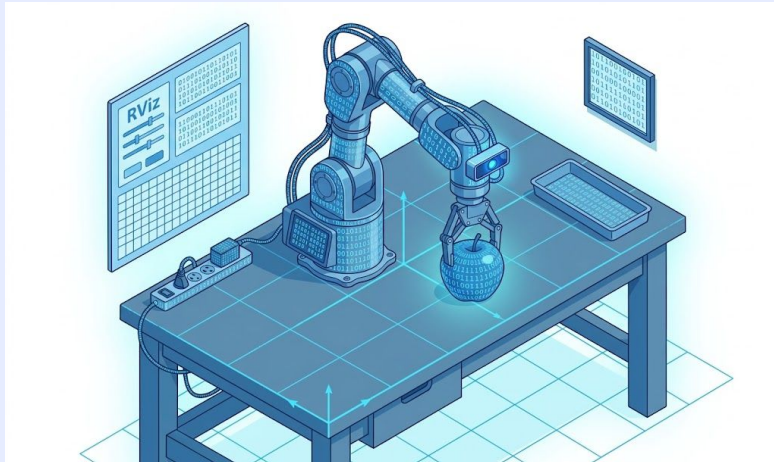
< Move it in Simulation, then make it real />

```
ros2 topic pub --once /joint_command sensor_msgs/msg/JointState '{name: [shoulder_pan], position: [0.5]}'
```

SO101 - Simulation

MuJoCo

- All your code will be tested here
- Safe



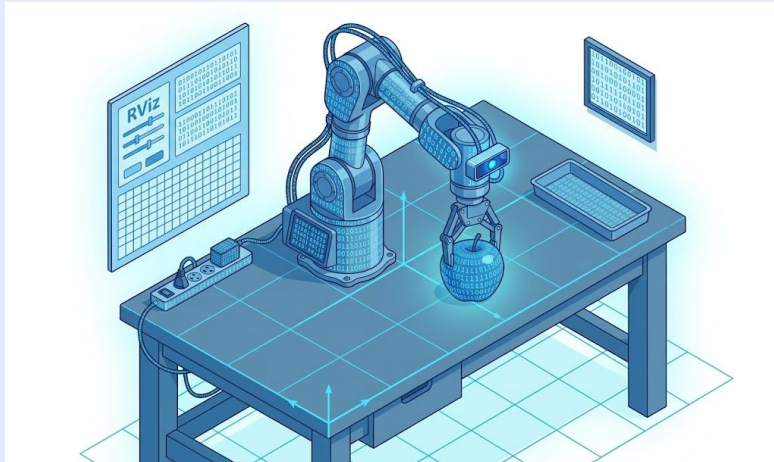
< Move it in Simulation, then make it real />

```
ros2 topic pub --once /joint_command sensor_msgs/msg/JointState '{name: [shoulder_pan], position: [0.5]}'
```

SO101 - Simulation

MuJoCo

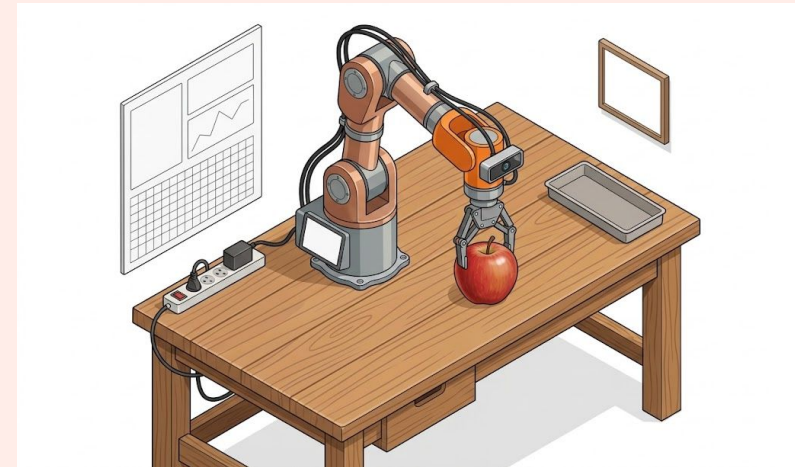
- All your code will be tested here
- Safe



SO101 - Real

The real arm

- Be careful (about the arm but also the screens and laptop around)
- Real validation of your work (and fun part of this project)



Let's continue



< Let's Dive into the Driver Node />



< Yesterday tips />

Changer le ROS_DOMAIN_ID dans .env :

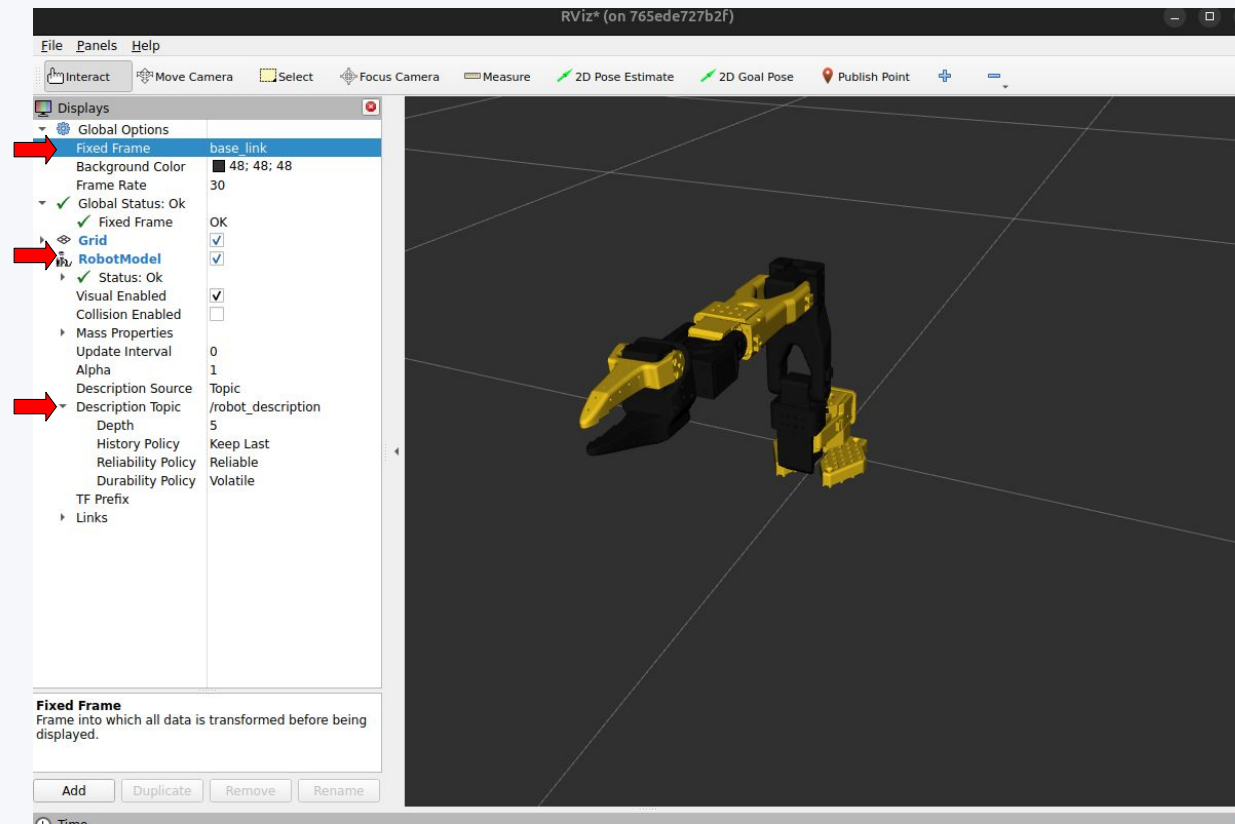
42 → Mettre un numéro entre 2 et 99 (votre département ou autre)

Lancer Rviz :

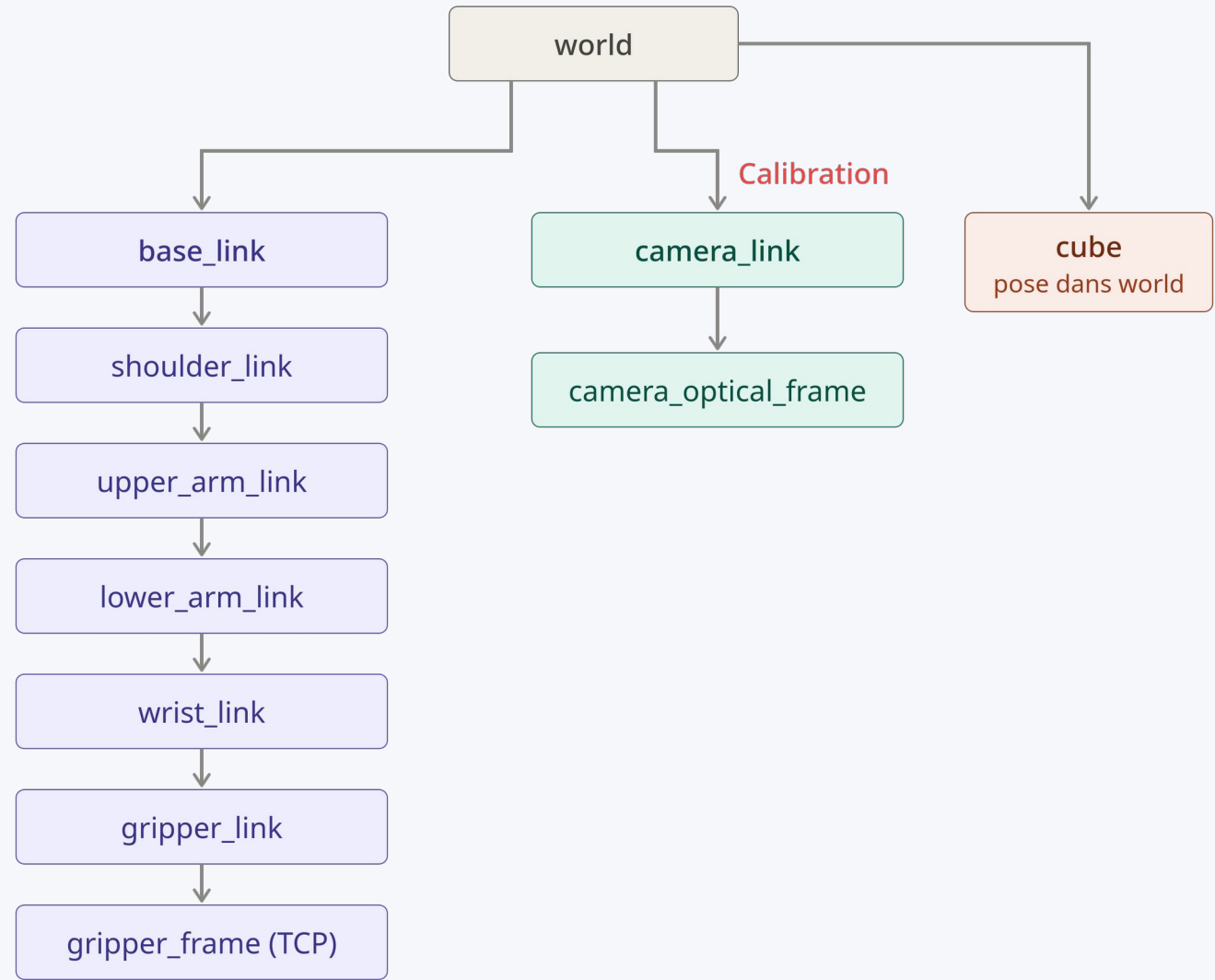
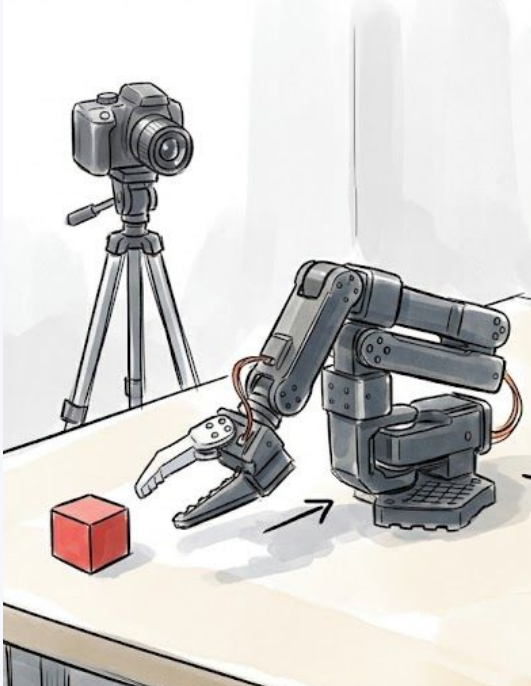
`docker compose exec control bash -lc "source /ros2_ws/install/setup.bash && rviz2"`

→ Dans robotModel → description Topic, mettre le topic /robot_description (Robot en vrac car /joint_states pas là
=> Faire le driver_node.

Dans RVIZ :



< TF Tree />



□ SO101

□ Caméra

□ Cube (perception)

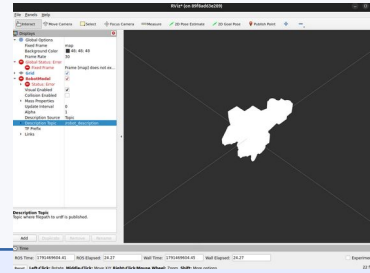
< Milestones />

1 Installation

Docker

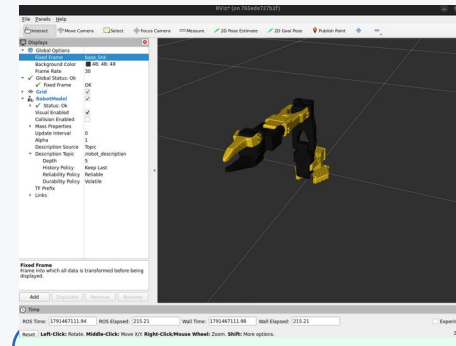
2 Rviz

Add model
Show the robot



3 Joint states

Show the joint states / TF
(Simulation)
Dans driver node , Var: self._robot



4 Real robot

Connect it
Move the robot in live and
see it in rviz

5 Publish joint command

1. Simulation
2. Real

6 Bonus Forward Kinematics

Calculate end-effector
position XYZ from joint states
(show it on rviz)

7 Bonus Inverse Kinematics

Move end-effector to XYZ
position.

8 Exploration

your application

< A quick explanation about the driver code />

driver_node.py

```
_connect_arm()
```

```
# Connect to the simulated (mujoco) or real arm
```

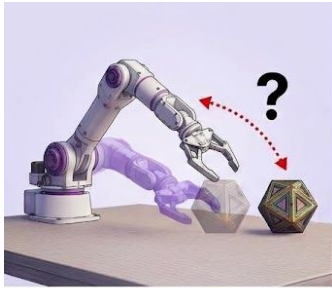
```
_publish_joint_states()
```

```
# Publish the joint states (to see the robot configuration on rviz!)
```

< brain node : is your code robust ? />

? The object moves mid-action

Recompute the new movement ?



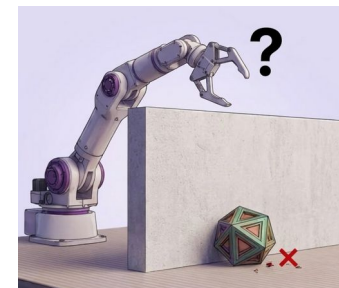
? The grasp fails

*How do you know?
What do you do?*



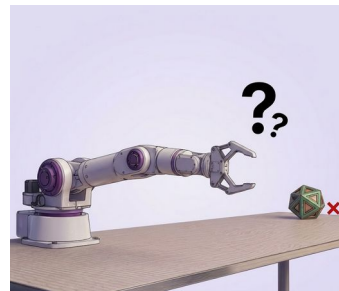
? The object leaves the view

*What does perception publish?
Is there a memory ?*



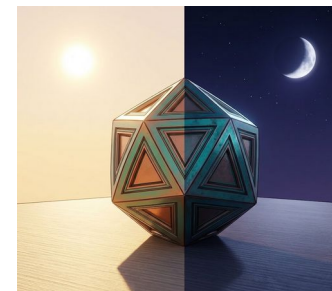
? Target out of reach

*What does IK return?
Does the arm move anyway?*



? Lighting changes

*Does your color threshold still hold?
Need a more robust perception ?*



{EPITECH}

<SINCE 1999/>

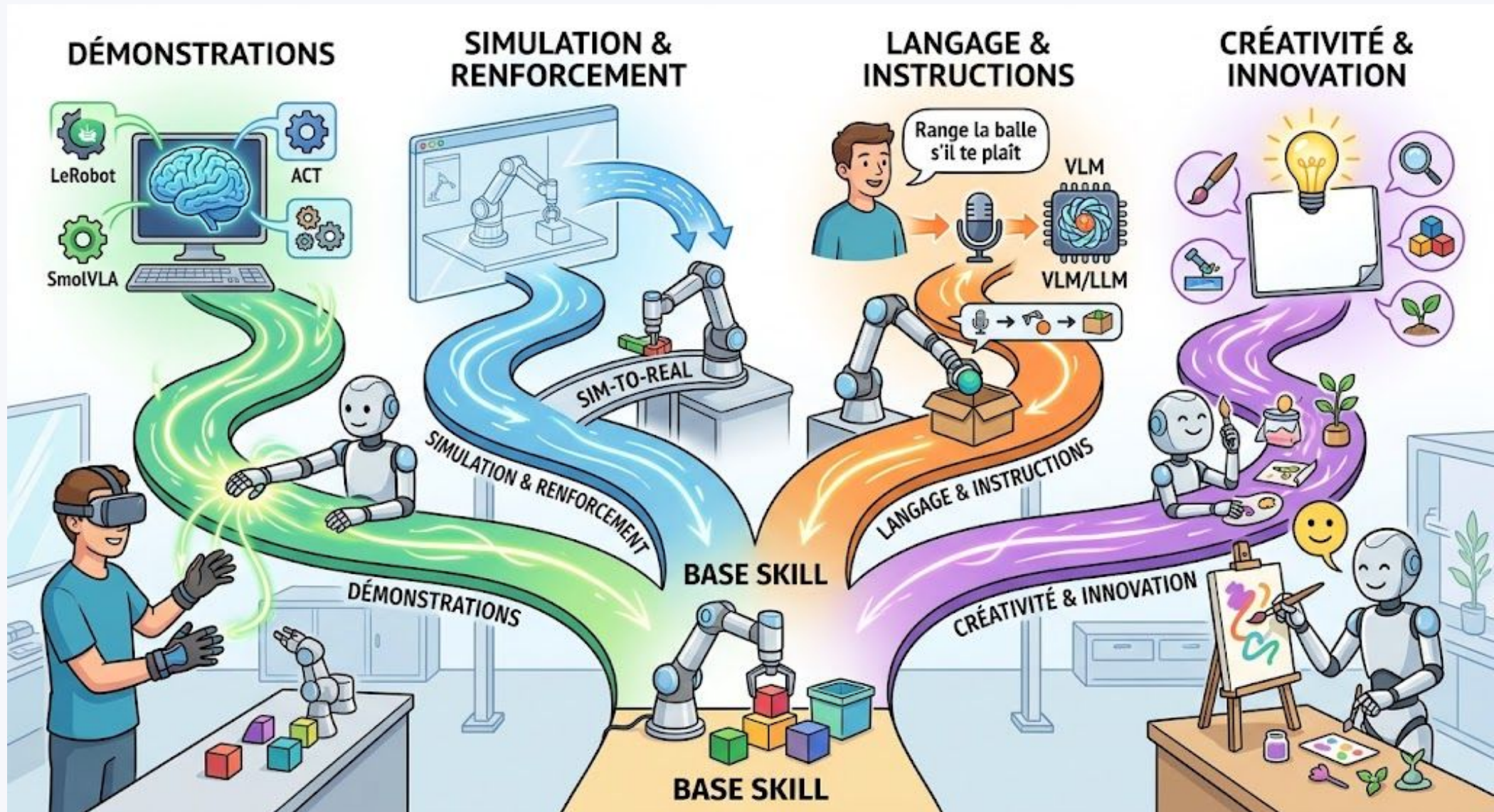
< Part 7 />

Going further

{ <TECH TOGETHER TOMORROW> }



< After pick & place />



< Workshop: find your application />

20~30

minutes, in groups

Your idea must be

- Feasible in a few weeks
- Demonstrable on video
- A real robotics skill

Your constraints

1 arm

5 DoF + gripper

Low payload

small, light objects

Small workspace

a tabletop

1 camera

+ whatever you add

Your code

ROS 2, sim first

Safety

near humans: slow

< check-in />

1

The idea

in one sentence

2

The tech

what to add to the pipeline

3

The risk

what might not work



Achievable examples

Color sorting · Tic-tac-toe vs the robot · Candy handout · Drawing · Tidying a workbench

Tools

Phase 1 is your basis + IA in control or perception / Add language command / Path or Motion planning

Be very ambitious !

Do something cool for you and your resume/cv !

If it does not work perfectly in the end, it's ok ;)